

Supporting Information:

Supporting Information: Electrostatic Grid and Surface Properties with Finite External Electric Fields in QUICK

Etienne Palos^{*,†} and TODO: Coauthors[‡]

[†]*Department of Chemistry, Michigan State University, East Lansing, Michigan, United States*

[‡]*TODO: Affiliations*

E-mail: TODO:email

Overview

This Supporting Information summarizes the CPU serial and MPI algorithms used for external-grid electrostatic potential (ESP), electric field (EFIELD), numerical electric field gradient (numerical EFG), analytic electric field gradient (analytic EFG), electron-density surface generation, and finite uniform external electric fields in QUICK. The grid-property and surface-property algorithms evaluate observables from a converged density, while the finite-field algorithm perturbs the SCF Hamiltonian and produces a field-polarized density for subsequent property analysis. These algorithms are the validated reference implementation for the present work. They also define the sign conventions, array shapes, and regression targets that should be preserved by future accelerator implementations. The convention used

by the grid-property implementation is

$$E_i(\mathbf{C}) = -\frac{\partial V(\mathbf{C})}{\partial C_i}, \quad G_{ij}(\mathbf{C}) = \frac{\partial E_i(\mathbf{C})}{\partial C_j}. \quad (1)$$

Thus the printed EFG tensor is the field-gradient tensor, or the negative of the Hessian of the electrostatic potential.

Algorithm S1: Electric Field on an External Grid

For each grid point $\mathbf{C}_g$, QUICK evaluates

$$\mathbf{E}(\mathbf{C}_g) = \mathbf{E}^{\text{nuc}}(\mathbf{C}_g) + \mathbf{E}^{\text{elec}}(\mathbf{C}_g). \quad (2)$$

The nuclear contribution is

$$E_i^{\text{nuc}}(\mathbf{C}_g) = \sum_A Z_A \frac{C_{gi} - R_{Ai}}{|\mathbf{C}_g - \mathbf{R}_A|^3}. \quad (3)$$

The implementation forms r^2 , evaluates $1/r$, and then constructs $1/r^3$ by multiplication. This avoids a general distance helper and repeated exponentiation.

The electronic contribution is evaluated shell pair by shell pair:

1. Loop over shell pairs (I, J) . In MPI, each rank owns a subset of outer shell indices.
2. Loop over primitive pairs (a, b) in the shell pair.
3. Apply the existing primitive-pair screening criterion.
4. Form the Gaussian product center $\mathbf{P}$ and primitive prefactor

$$K_{ab} = \frac{2\pi}{p} \exp \left[-\frac{ab}{p} |\mathbf{A} - \mathbf{B}|^2 \right], \quad p = a + b.$$

5. For each grid point, evaluate Boys auxiliaries and first derivative attraction-integral arrays.
6. Contract the derivative integrals with the density matrix to accumulate $\mathbf{E}^{\text{elec}}$.
7. In MPI, reduce the local electronic arrays over $3N_{\text{grid}}$ double-precision values.

Algorithm S2: Analytic Electric Field Gradient

The analytic EFG evaluates

$$G_{ij}^{\text{elec}}(\mathbf{C}_g) = \sum_{\mu\nu} P_{\mu\nu} \frac{\partial^2}{\partial C_i \partial C_j} (\mu | r_{C_g}^{-1} | \nu). \quad (4)$$

Only six independent tensor components are evaluated explicitly:

$$xx, \ xy, \ xz, \ yy, \ yz, \ zz,$$

and the symmetric partners are filled by equality.

For an s -type primitive base integral,

$$\frac{\partial^2 I^{(m)}}{\partial C_i^2} = -2pI^{(m+1)} + 4p^2(P_i - C_i)^2 I^{(m+2)}, \quad (5)$$

$$\frac{\partial^2 I^{(m)}}{\partial C_i \partial C_j} = 4p^2(P_i - C_i)(P_j - C_j) I^{(m+2)} \quad i \neq j. \quad (6)$$

Higher angular momentum is generated by the Obara-Saika recurrence over Cartesian exponents on the two Gaussian centers.

The optimized CPU organization is:

1. Loop over MPI-owned shell pairs.
2. Loop over primitive pairs and screen them.

3. Compute $\mathbf{P}$, p , and K_{ab} directly.
4. Process grid points in blocks of 128 points.
5. For each point in the block, compute and store Boys auxiliaries $I^{(m)}$ through the maximum order needed by the second derivatives.
6. Traverse angular shells, basis-function pairs, density elements, and contraction coefficients once for the block.
7. For each basis-function pair and grid point in the block, evaluate the six recursive second derivatives and accumulate the nine printed tensor components.
8. In MPI, reduce the local electronic tensor over $9N_{\text{grid}}$ double-precision values.

The block size is intentionally modest. A 128-point block stores 21×128 auxiliary double-precision values, which is small enough for cache locality while reducing repeated density and coefficient traversal relative to a one-grid-point contraction.

Algorithm S3: Numerical Electric Field Gradient

The numerical EFG remains as a reference path and is selected by the `EFG_GRID_NUMERICAL` keyword. For a displacement $h = 10^{-4}$ bohr,

$$G_{ij}^{\text{num}}(\mathbf{C}_g) = \frac{E_i(\mathbf{C}_g + h\mathbf{e}_j) - E_i(\mathbf{C}_g - h\mathbf{e}_j)}{2h}. \quad (7)$$

The implementation reuses a single displaced-coordinate work array for the plus and minus field evaluations. This reduces temporary memory and grid-copy traffic relative to storing separate plus and minus coordinate arrays.

Algorithm S4: Nuclear EFG

The nuclear and external point-charge contribution to the printed EFG tensor is

$$G_{ij}^{\text{nuc}}(\mathbf{C}_g) = \sum_A Z_A \left[\frac{\delta_{ij}}{r_A^3} - \frac{3(C_{gi} - R_{Ai})(C_{gj} - R_{Aj})}{r_A^5} \right], \quad r_A = |\mathbf{C}_g - \mathbf{R}_A|. \quad (8)$$

The CPU kernel forms r^2 , $1/r$, $1/r^3$, and $1/r^5 = (1/r^3)/r^2$, then fills the tensor components directly.

Algorithm S5: Finite Uniform External Electric Field

For an applied field $\mathbf{F}$ and origin $\mathbf{r}_0$, QUICK uses the scalar potential

$$\phi_F(\mathbf{r}) = -\mathbf{F} \cdot (\mathbf{r} - \mathbf{r}_0). \quad (9)$$

The electronic one-electron contribution added to Hcore is

$$h_{\mu\nu}^F = \sum_{i=x,y,z} F_i \langle \chi_\mu | r_i - r_{0i} | \chi_\nu \rangle. \quad (10)$$

The classical charge-field energy is

$$E_{\text{core}}^F = - \sum_A Z_A \mathbf{F} \cdot (\mathbf{R}_A - \mathbf{r}_0) - \sum_q q \mathbf{F} \cdot (\mathbf{R}_q - \mathbf{r}_0), \quad (11)$$

where the second sum is included only when external point charges are present.

The CPU algorithm is:

1. Parse `EXTERNAL_EFIELD_X/Y/Z` or `FINITE_FIELD_X/Y/Z`; the components are in atomic units.
2. Optionally parse origin keywords. The origin is in bohr and defaults to (0,0,0).

3. During one-electron matrix construction, loop over owned basis-function rows.
4. For each basis pair, evaluate the first moment integrals about $\mathbf{r}_0$ using QUICK's existing Obara-Saika moment recurrence.
5. Add $F_x M_x + F_y M_y + F_z M_z$ to the lower triangle of the one-electron matrix.
6. Symmetrize and store the result in the reusable Hcore array.
7. Add E_{core}^F to the classical core energy before the SCF energy loop.

The implementation currently supports SCF energies and post-SCF property analysis. Analytic gradients under an external electric field require derivatives of the moment integrals and the classical charge-field force term, and are intentionally not enabled in the present implementation.

Algorithm S6: Electron-Density Isosurface Properties

The electron-density surface route first generates a point cloud from the converged density and then passes the points to the same OEPROP kernels used for external grids. The density at a point $\mathbf{C}$ is evaluated as

$$\rho(\mathbf{C}) = \sum_{\mu\nu} P_{\mu\nu} \chi_{\mu}(\mathbf{C}) \chi_{\nu}(\mathbf{C}) \quad (12)$$

for restricted calculations. For unrestricted calculations, the alpha and beta density matrices are summed:

$$\rho(\mathbf{C}) = \sum_{\mu\nu} (P_{\mu\nu}^{\alpha} + P_{\mu\nu}^{\beta}) \chi_{\mu}(\mathbf{C}) \chi_{\nu}(\mathbf{C}). \quad (13)$$

For a requested isovalue ρ_0 , the current CPU algorithm is:

1. Parse `ESP_SURFACE`, `EFIELD_SURFACE`, or `EFG_SURFACE`. These keywords request an electron-density surface and select the corresponding property evaluator.

2. Parse optional controls for the isovalue, approximate point spacing, and maximum ray length.
3. Generate a set of approximately uniform atom-centered ray directions.
4. For each atom and ray, start at the atom center and step outward until $\rho(\mathbf{C}) \leq \rho_0$, or until the maximum ray length is reached.
5. If a crossing is found, refine the interval by bisection to locate the first outward crossing of $\rho(\mathbf{C}) = \rho_0$.
6. Add the point to the surface point cloud only if it is not already represented within a spacing-dependent duplicate threshold.
7. In MPI, generate the surface on the master rank and broadcast the point count and coordinates to all ranks.
8. Call the existing ESP, EFIELD, or analytic EFG grid evaluator with the generated coordinates.

This algorithm is a deterministic point-cloud sampler. It does not construct a triangulated molecular surface or surface area weights. Its purpose in the present implementation is to place ESP, EFIELD, and EFG on a chemically meaningful isodensity shell using the same property kernels and MPI reductions as external-grid calculations.

MPI Parallelism

MPI parallelism follows the existing QUICK shell-pair distribution. Each rank receives a set of outer shell indices and computes all corresponding (I, J) shell pairs with $J \geq I$. Nuclear contributions are inexpensive and are evaluated redundantly. Electronic contributions are reduced to the master rank:

Property	Local electronic array	MPI reduction length
EFIELD	$3 \times N_{\text{grid}}$	$3N_{\text{grid}}$
EFG	$3 \times 3 \times N_{\text{grid}}$	$9N_{\text{grid}}$

For electron-density surface calculations, the master rank generates the isosurface point cloud and broadcasts the coordinates. After that broadcast, surface calculations use the same shell-pair decomposition and electronic-array reductions as ordinary external-grid calculations.

Finite external fields use the existing MPI one-electron matrix distribution rather than the OEPROP shell-pair reduction. Each rank adds the field operator only for its owned basis-function rows, in the same ownership pattern used for kinetic-energy one-electron integrals. The resulting one-electron matrices are then handled by the existing SCF/Fock reductions. This avoids multiplying the external-field operator by the number of MPI ranks.

Keyword and Input Examples

External-grid properties are requested by adding one or more property keywords to the normal QUICK keyword line. For example:

```
PBE0 D3BJ BASIS=cc-pVDZ cutoff=1.0e-12 denserms=4.0e-8
ESP_GRID EFIELD_GRID EFG_GRID
```

```
C    0.0    0.0    0.0
O    1.22731534 -0.00000104 -0.00201325
...

0.4963301661    0.6476371381    2.2357599387
0.4963301661   -0.3424539632    2.3023426438
...
```


A finite-field single point is requested on the keyword line. QUICK also supports multiple keyword lines, which can keep field inputs readable:

```
HF BASIS=STO-3G cutoff=1.0e-9 denserms=1.0e-6
ENERGY DIPOLE CHARGE=0 EXTERNAL_EFIELD_X=1.0E-4
```

```
O  -0.33840   0.00380   0.23923
H  -0.33510  -0.00190  -0.83277
H   0.67350  -0.00190   0.59353
```

The alias `FINITE.FIELD_X=1.0E-4` gives the same result. Optional origin keywords, for example `EXTERNAL_EFIELD.ORIGIN.Z=1.0`, set the origin for both the one-electron moment operator and the classical charge-field energy. For charged systems, the finite-field energy depends on this origin; the default is the Cartesian origin in bohr.

Electron-density surface properties are requested by surface keywords:

```
HF BASIS=STO-3G cutoff=1.0e-9 denserms=1.0e-6 ENERGY CHARGE=0
ESP_SURFACE EFIELD_SURFACE EFG_SURFACE DENSITY_SURFACE_VALUE=1.0E-3
DENSITY_SURFACE_SPACING=1.25 DENSITY_SURFACE_MAX_RADIUS=6.0
```

```
O  -0.33840   0.00380   0.23923
H  -0.33510  -0.00190  -0.83277
H   0.67350  -0.00190   0.59353
```

The surface value is in atomic units. The spacing and maximum ray length are in angstroms, matching the coordinate convention used for generated molecular-surface output.

GPU OEPROP Implementation and Validation Protocol

QUICK already contains a CUDA/HIP electronic ESP path for external-grid OEPROP calculations. The Fortran OEPROP layer uploads the external grid coordinates and the converged density matrix, the GPU kernel evaluates the electronic attraction contribution shell-pair by shell-pair, and the CPU layer then adds the direct nuclear ESP. The same separation is preserved for the GPU EFIELD and EFG extension: nuclear EFIELD and EFG remain CPU-side direct inverse-distance kernels, while the density-contracted electronic contribution is evaluated on the accelerator. Keeping nuclear terms on the host also keeps the first validation target narrow: CPU and GPU must agree for the electronic density-contracted attraction derivatives, while the inexpensive classical point-charge expressions are shared with the CPU reference implementation.

The GPU electronic arrays use the same component-fast memory convention as the CPU and MPI reductions. EFIELD is stored as $3N_{\text{grid}}$ values ordered (E_x, E_y, E_z) for each point. EFG is stored as $9N_{\text{grid}}$ values ordered according to the Fortran array $G(3, 3, N_{\text{grid}})$, namely

$$(G_{xx}, G_{yx}, G_{zx}, G_{xy}, G_{yy}, G_{zy}, G_{xz}, G_{yz}, G_{zz})$$

for each grid point. For an electrostatic field gradient the off-diagonal components are symmetric, but both positions are filled explicitly to match the printed CPU tensor.

The host/device algorithm is:

1. Build the usual SCF density on the CPU or GPU-enabled QUICK path.
2. Allocate a device grid-coordinate array of shape $3 \times N_{\text{grid}}$.
3. Allocate one electronic output buffer: N_{grid} for ESP, $3N_{\text{grid}}$ for EFIELD, or $9N_{\text{grid}}$ for EFG.

4. Upload the alpha density matrix, and upload the beta density matrix for unrestricted calculations.
5. Copy the OEPROP simulation descriptor and Cartesian angular-momentum lookup table to GPU constant memory.
6. Launch the ESP, EFIELD, or EFG GPU kernel for the requested property.
7. Download the electronic property buffer to the Fortran layer.
8. In MPI-GPU builds, reduce the electronic buffer over ranks using the same lengths as the CPU MPI implementation.
9. Add the CPU nuclear contribution and write or return the total property.

The GPU kernels use the existing ESP work decomposition. A global one-dimensional work index spans

$$N_{\text{work}} = N_{\text{shell}}^2 N_{\text{grid}}.$$

Each thread walks this index space with a grid-stride loop. The work index is decoded into a shell-pair index and a grid-point index. The sorted shell-pair arrays and the $J \geq I$ condition select the unique shell pairs; in MPI-GPU builds the same shell-pair ownership mask used elsewhere in QUICK suppresses work not owned by the current rank. For the selected shell pair and grid point, the kernel loops over primitive pairs, applies the existing one-electron screening, builds the Gaussian product center and attraction prefactor, evaluates the requested attraction derivative integrals, contracts them with the density matrix and Cartesian normalization factors, and atomically accumulates the result into the component-fast output buffer. The off-diagonal density factor is two, matching the lower-triangular shell-pair contraction used by the CPU ESP implementation.

The three GPU kernels differ only in the integral quantity accumulated. The ESP kernel keeps the historical sign of the scalar electronic potential contribution. The EFIELD and

EFG kernels use positive attraction auxiliaries and then form derivatives with respect to the probe coordinate $\mathbf{C}$. The electronic EFIELD evaluates

$$E_i^{\text{elec}}(\mathbf{C}) = \sum_{\mu\nu} P_{\mu\nu} \frac{\partial}{\partial C_i} (\mu | r_C^{-1} | \nu),$$

which is the electronic contribution to $E_i = -\partial V / \partial C_i$. The electronic EFG evaluates

$$G_{ij}^{\text{elec}}(\mathbf{C}) = \frac{\partial E_i^{\text{elec}}(\mathbf{C})}{\partial C_j},$$

not a finite-difference approximation and not the opposite potential-Hessian sign convention. In the present reference implementation, these first and second derivatives are obtained from a device recurrence that mirrors the CPU center-derivative formulas. For s -type base integrals the recurrence differentiates the Boys auxiliary level directly; for higher angular momentum it applies the same Obara-Saika center-transfer structure used by the CPU analytic evaluator. This conservative analytic path is intended to establish sign and component correctness before replacing the recursive device evaluator with generated or iterative derivative recurrence tables.

The reference GPU EFG path is therefore an analytic implementation, but not yet the final performance implementation. The expected optimized version should generate derivative-specific recurrence tables for the six independent second derivatives (xx, xy, xz, yy, yz, zz) , vectorize over grid points within a primitive pair, and reduce atomic-update pressure by using block-level partial sums where grid sizes justify the additional shared-memory traffic. The acceptance criterion for that future optimized kernel is bitwise or near-bitwise agreement with the current CPU analytic EFG and the reference GPU analytic EFG on the validation suite below.

The local Apple Silicon development machine cannot compile or run CUDA/HIP kernels. GPU validation is therefore defined as an external-host protocol. On a CUDA or HIP machine, build one CPU QUICK executable and one GPU QUICK executable, then run:

```
export CPU_QUICK=/path/to/cpu/install/bin/quick
export GPU_QUICK=/path/to/gpu/install/bin/quick
export QUICK_BASIS=/path/to/install/basis
validation/gpu_oeprop/validate_gpu_oeprop.sh
```

The script runs the acetone B3LYP/def2-SVP ESP, EFIELD, analytic EFG, and numerical EFG grid regressions with both executables, compares CPU and GPU property files, and compares GPU analytic EFG against GPU numerical EFG. The acceptance target is a maximum absolute property difference of 2×10^{-7} a.u. for the printed property files, consistent with the existing OEPROP regression tolerance.

Finite external fields remain a separate accelerator target because the perturbation is a one-electron matrix contribution. A GPU path could either compute the moment-integral matrix on CPU and upload the perturbed Hcore, or implement the first-moment integral build directly on device. The CPU analytic and numerical EFG agreement, the serial/MPI finite-field energy agreement, and the GPU OEPROP validation protocol above provide the acceptance targets for this later work.

Trajectory and AIMD/BOMD Property Analysis

The current CPU implementation can evaluate ESP, EFIELD, and EFG after any converged single-point or gradient calculation. This is enough for trajectory post-processing: for each stored geometry, run a QUICK single point with ESP_GRID, EFIELD_GRID, and/or EFG_GRID, using either a fixed external probe grid or a geometry-dependent grid generated by a pre-processing script. QUICK can also store optimization trajectories in the checkpoint file and read a selected optimization step back as a single-point geometry, which gives a practical route to property evaluation along optimization paths.

The library interface now also exposes a no-I/O OEPROP wrapper:

```
call getQuickOEPROP(npoints, probe_xyz_bohr, ierr, &
```

```
esp=esp, efield=efield, efg=efg)
```

The probe coordinates are in bohr. The optional output array shapes are:

```
esp(npoints)
```

```
efield(3,npoints)
```

```
efg(3,3,npoints)
```

The routine uses the current converged density from a preceding energy or gradient API call and delegates all property evaluation to the existing OEPROP evaluator; it does not duplicate the integral code. It returns a clean error if called before a density is available, if no property output is requested, or if the number of probe points is invalid.

A lightweight trajectory driver, `test-api-oeprop`, exercises this API in two modes. The `validate` mode runs a small water RHF/STO-3G calculation, evaluates ESP, EFIELD, and EFG on four fixed probe points, and writes a compact CSV file. The `trajectory` mode reads a multi-frame XYZ trajectory and a probe-point file in angstroms, converts probe points to bohr for `getQuickOEPROP`, reuses the QUICK density between frames when possible, and writes one CSV row per frame and probe point.

The trajectory CSV columns are:

```
frame,probe,energy_hartree,x_ang,y_ang,z_ang,esp,
efield_x,efield_y,efield_z,efield_mag,
efg_xx,efg_xy,efg_xz,efg_yx,efg_yy,efg_yz,efg_zx,efg_zy,efg_zz,
efg_trace,efg_frobenius,efg_anisotropy
```

The anisotropy is computed as the rotationally invariant deviatoric norm,

$$\eta_G = \left[\frac{3}{2} \sum_{ij} \left(G_{ij} - \frac{\text{Tr}(G)}{3} \delta_{ij} \right)^2 \right]^{1/2},$$

which is equivalent to the corresponding eigenvalue expression for the symmetric field-gradient tensor.

For AIMD or BOMD studies of local electrostatics, the recommended staged workflow is therefore:

1. First use trajectory post-processing on a saved external trajectory. This validates grid definitions, output sizes, signs, units, and time-series analysis without touching the force loop.
2. Use the `getQuickOEPROP` library wrapper from an external MD or QM/MM driver to request properties on the fly after each SCF/gradient call.
3. Only then add a native QUICK BOMD loop, if desired. A native loop would need velocity initialization, an integrator, thermostat/barostat choices, restart files, and energy-conservation diagnostics. Those pieces are separate from the OEPROP kernels themselves.

With this staging, the CPU OEPROP implementation is suitable for post-SCF trajectory analysis and callable on the fly through the API. QUICK should not yet be described as having a complete native AIMD/BOMD engine with OEPROP emitted automatically at every step.

OEPROP API Validation

The no-I/O API was validated by comparing `getQuickOEPROP` against the standard file-producing `ESP_GRID`, `EFIELD_GRID`, and `EFG_GRID` paths for water RHF/STO-3G on four fixed probe points. The validation driver also checks that calling `getQuickOEPROP` before a converged density returns the expected clean error. The trajectory mode was tested on a three-frame water XYZ trajectory with the same four probe points; the first trajectory frame is identical to the validation geometry and therefore provides a direct trajectory-vs-validation check.

Comparison	Values	Max. abs. dev. / a.u.
File ESP vs API ESP	4	3.66×10^{-11}
File EFIELD vs API EFIELD	12	4.71×10^{-9}
File EFG vs API EFG	36	3.85×10^{-9}
Trajectory frame 1 ESP vs validation	4	0.0
Trajectory frame 1 EFIELD vs validation	12	0.0
Trajectory frame 1 EFG vs validation	36	0.0

The existing density-surface OEPROP regression subset was also rerun locally after adding the API wrapper. The ESP, EFIELD, and EFG density-surface property files were bitwise identical to the saved references for the small water RHF/STO-3G tests.

Validation

The optimized implementation was checked against an acetone 663-point external-grid regression case. The method and basis were PBE0-D3BJ and cc-pVDZ. The serial and MPI ESP/OEPROP regression subsets passed with zero failures. Analytic and numerical EFG tensors agree to approximately 10^{-9} a.u. at printed precision.

Finite external fields were validated with a water RHF/STO-3G regression input using $F_x = 10^{-4}$ a.u. and $\mathbf{r}_0 = \mathbf{0}$. The serial calculation produced

$$E_{\text{tot}} = -74.947913548 E_h, \quad (14)$$

and the two-rank MPI calculation produced the same value at printed precision. The zero-field and $\pm 10^{-4}$ a.u. field calculations gave

$$\frac{E(+F_x) - E(-F_x)}{2F_x} \approx -0.497155 \text{ a.u.}, \quad (15)$$

which agrees with $-\mu_x$ from the zero-field printed dipole,

$$-\frac{1.2637 \text{ D}}{2.541765} \approx -0.49717 \text{ a.u.} \quad (16)$$

Electron-density surface properties were validated with water RHF/STO-3G using $\rho_0 = 10^{-3}$ a.u., a spacing of 1.25 Å, and a maximum ray length of 6.0 Å. The sampler generated 62 points. Serial and two-rank MPI calculations gave the same total energy,

$$E_{\text{tot}} = -74.947863811 E_h, \quad (17)$$

and the generated ESP, EFIELD, and analytic EFG property files were bitwise identical between serial and MPI runs.

Independent PySCF Validation

Cross-code validation was performed with PySCF 2.6.2 using the Anaconda Python interpreter at `/opt/anaconda3/bin/python`. PySCF molecules were built with the same Cartesian coordinates, total charge, basis names, and Cartesian Gaussian basis convention used by QUICK. The Cartesian convention is important for the acetone cc-pVDZ case: QUICK uses 90 Cartesian AO functions, matching PySCF with `cart=True`; PySCF’s default spherical representation would give 86 AO functions.

The PySCF ESP at each point was evaluated as

$$V^{\text{PySCF}}(\mathbf{C}) = \sum_A \frac{Z_A}{|\mathbf{C} - \mathbf{R}_A|} - \sum_{\mu\nu} P_{\mu\nu} \langle \chi_\mu | r_C^{-1} | \chi_\nu \rangle. \quad (18)$$

PySCF EFIELD and EFG reference values were then obtained by finite differences of this

ESP with $h = 5 \times 10^{-4}$ bohr:

$$E_i^{\text{PySCF}}(\mathbf{C}) = -\frac{V(\mathbf{C} + h\mathbf{e}_i) - V(\mathbf{C} - h\mathbf{e}_i)}{2h}, \quad (19)$$

$$G_{ij}^{\text{PySCF}}(\mathbf{C}) = -\frac{\partial^2 V(\mathbf{C})}{\partial C_i \partial C_j}. \quad (20)$$

The diagonal second derivatives used the standard three-point central stencil, and the off-diagonal terms used the central mixed-derivative stencil. This gives an independent check of QUICK’s sign convention because the PySCF reference starts from the scalar ESP rather than QUICK’s EFIELD or EFG kernels.

Comparison	Values	Max. abs. dev. / a.u.	RMS dev. / a.u.
Acetone ESP grid	663	4.110×10^{-6}	8.528×10^{-7}
Acetone EFIELD grid	1989	2.400×10^{-6}	2.974×10^{-7}
Acetone EFG grid	5967	1.510×10^{-6}	1.689×10^{-7}
Water ESP density surface	62	2.582×10^{-7}	1.438×10^{-7}
Water EFIELD density surface	186	3.123×10^{-8}	1.108×10^{-8}
Water EFG density surface	558	4.114×10^{-8}	1.136×10^{-8}

For the acetone PBE0-D3BJ/cc-pVDZ calculation, the local PySCF installation was used for the matching PBE0 density. QUICK’s printed D3BJ dispersion correction was $-0.004917980 E_h$. This term was subtracted from the QUICK total energy for the SCF energy comparison because D3BJ is a post-SCF geometry-dependent correction and does not alter the one-particle density. The resulting PySCF and QUICK non-dispersion PBE0 total energies differed by $1.385 \times 10^{-5} E_h$. The few- 10^{-6} a.u. property differences are attributed to small cross-code DFT quadrature and SCF-density differences. For water RHF/STO-3G, the PySCF and QUICK zero-field total energies differed by only $2.149 \times 10^{-8} E_h$, and the surface-property deviations were correspondingly much smaller.

The finite external electric field was validated by applying $F_x = \pm 10^{-4}$ a.u. to water RHF/STO-3G in both codes. In PySCF, the comparison Hamiltonian was constructed by

adding

$$\sum_i F_i (\langle \chi_\mu | r_i | \chi_\nu \rangle - r_{0i} S_{\mu\nu}) \quad (21)$$

to the one-electron Hamiltonian and adding the same classical charge-field energy as QUICK. The $F_x = +10^{-4}$ a.u. total energies differed by $2.158 \times 10^{-8} E_h$, and the central finite-field derivative differed by 1.103×10^{-6} a.u.

Optimization Options for Future Work

The current analytic EFG implementation is mathematically analytic but still uses a general recursive evaluator for second derivatives. Further optimization should focus on replacing that recursion with generated or iterative recurrence tables for the six independent second derivatives, analogous to QUICK’s optimized first-derivative EFIELD arrays. Additional options include:

- prepacking density-weighted shell-pair contraction coefficients for repeated grid use;
- vectorizing over grid points inside a fixed primitive and angular-momentum class;
- exposing the grid-block loop to OpenMP or another shared-memory threading layer;
- evaluating whether GPU kernels are worthwhile for large external grids or density surfaces;
- replacing the current radial-ray density-surface point cloud with a higher-quality sampler or triangulated isosurface when surface weights are needed;
- caching Cartesian moment matrices for repeated finite-field scans over different field vectors;
- adding tensor-invariant output, including trace, Frobenius norm, eigenvalues, and anisotropy measures.